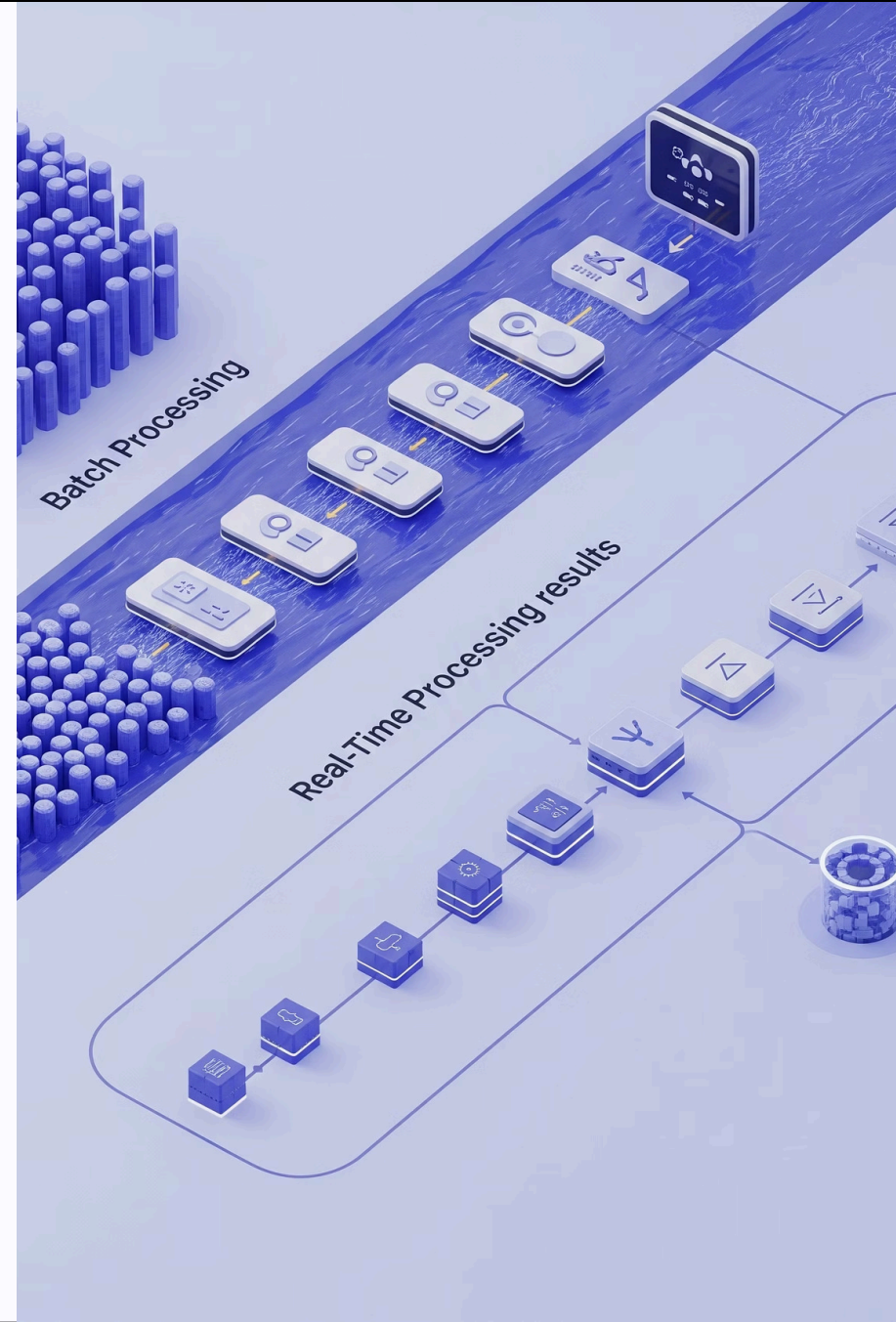


Batch Inference vs Online Inference

Choosing the Right Serving Mode for Your ML Workload

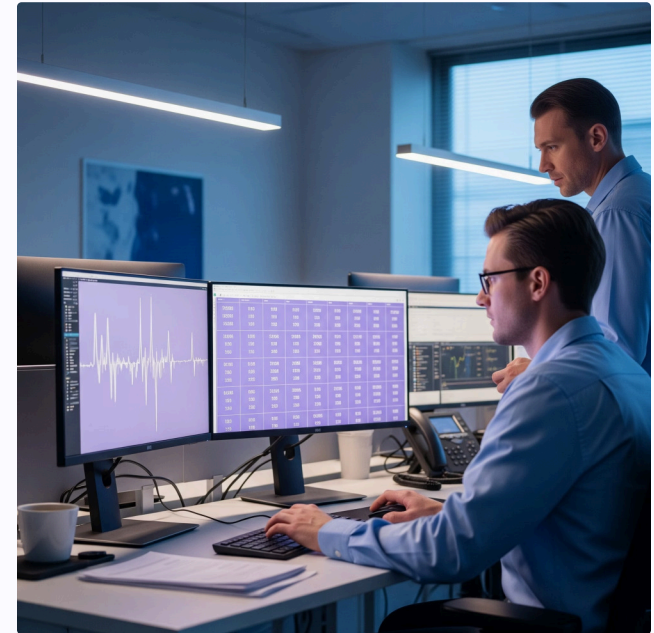
A strategic guide for ML engineers and technical product managers to optimize inference workloads.



Why This Matters

The way you serve ML predictions impacts everything from user experience to infrastructure costs. Making the right choice means understanding trade-offs between:

- Inference workloads aren't one-size-fits-all
- Mode affects latency, cost, infra design
- Decision = match workload pattern to serving strategy



Batch Inference

Batch inference processes predictions in bulk rather than individually, optimizing for efficiency over immediacy.

Large volumes of predictions processed in **bulk jobs**

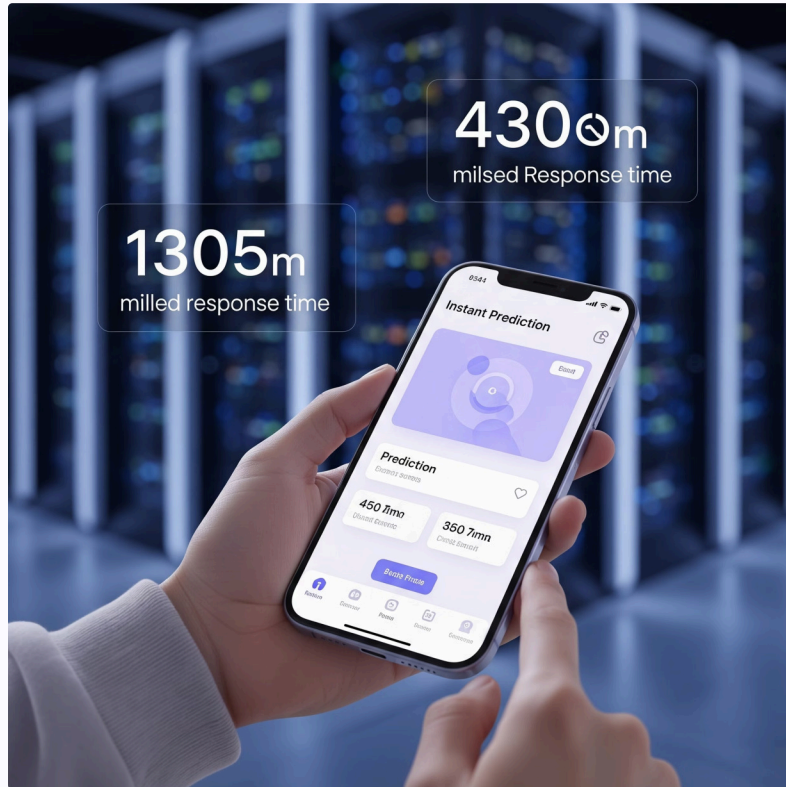
Typically run on a **schedule** (hourly, nightly, weekly)

Optimized for **throughput and cost efficiency**

Examples: churn prediction, risk scoring, recommendation refresh



Online Inference



Online inference delivers predictions in real-time as users or systems request them, prioritizing speed and availability.

Predictions served **on-demand** in real time

Must meet strict **p95/p99 latency SLOs**

Optimized for **responsiveness and availability**

Examples: fraud detection, ads ranking, chatbot responses

Batch Inference Characteristics



High throughput

Process millions of predictions in one job with relaxed latency requirements



Large clusters

Leverages distributed computing frameworks (Spark, Ray, Airflow, Kubeflow)



Result storage

Store predictions in databases, caches, or feature stores for later retrieval



Cost efficiency

Often cheaper due to ability to leverage spot/preemptible instances



Online Inference Characteristics



Low latency/jitter

Critical response times measured in milliseconds with consistent performance



Needs autoscaling

Infrastructure must dynamically adjust to absorb traffic spikes



REST/gRPC APIs

Typically served through standardized API protocols for integration



Modern infrastructure

Kubernetes + GPU/CPU optimized nodes + sophisticated load balancing





Infrastructure Trade-Offs

Batch

Cost efficient for non-urgent jobs

Pay only for compute when batch jobs run, not 24/7

Fault tolerance

Easy to parallelize; tolerant of retries and failures

Resource optimization

Can utilize resources during off-peak hours

Online

Higher operational costs

More expensive due to 24/7 infrastructure footprint

Reliability engineering

Requires HA, sophisticated monitoring, and resilience

Complex scaling

Must handle unpredictable demand patterns

Hybrid Patterns

Many sophisticated ML systems combine both approaches to maximize benefits while minimizing drawbacks.

Precompute + serve online

Generate recommendations in batch, serve from cache in real-time



Batch warmup + online refinement

NLP embeddings cached in batch, then query-specific reranking in real-time

Streaming micro-batch

Kafka/Flink processing bridging both modes with near-real-time capabilities

Common Pitfalls

- Using **online inference** when batch suffices → overspending

Real-time services are expensive. Don't use them for predictions that could be precomputed.

- Ignoring **feature skew** between offline batch vs online service

Different processing pipelines can lead to inconsistent feature computation.

- Not monitoring **tail latency** in real-time APIs

p99 latency spikes can ruin user experience even when averages look good.

- Overloading online infra with requests better suited for batch

Causing unnecessary scaling events and higher costs.



Decision Matrix

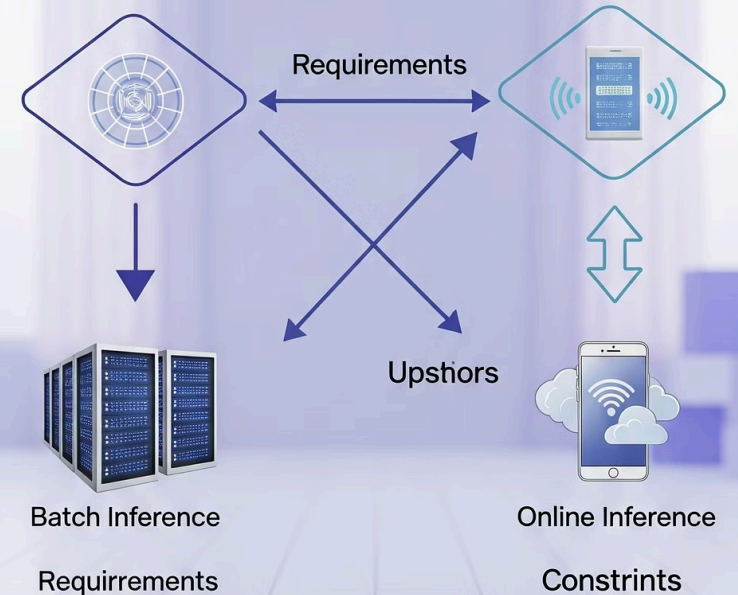
Your ML Inference Cheat Sheet

Choose **Batch** When:

- Workload tolerates latency of minutes → hours
- Throughput + cost > real-time needs
- Predict once, reuse many times
- Processing window is predictable
- Resource efficiency is prioritized

Choose **Online** When:

- Requires response in ms → sec
- Latency + SLA-driven workloads
- Predictions tied to user interactions
- Immediate decisions are needed
- User experience depends on speed



Best Practices

01

Define **SLOs** before choosing mode

Establish clear service level objectives for latency, throughput, and availability to guide architectural decisions

02

Use **feature stores** for consistency

Maintain consistent feature computation between batch and online systems with a centralized feature repository

03

Monitor both **accuracy drift** and **infra metrics**

Track model performance alongside system health to catch both ML and operational issues

04

Design **clear ownership** between batch + online pipelines

Establish team responsibilities for hybrid architectures to prevent operational gaps





Key Takeaways

Batch = throughput, cost, offline jobs

Optimize for efficiency when immediate results aren't required

Many enterprises use **hybrid architectures**

Combine approaches to get the best of both worlds

Online = latency, responsiveness, real-time systems

Prioritize speed and availability for user-facing applications

Right choice = align serving mode with **business requirements**

Let your use case drive the technology decisions, not vice versa

Questions? Contact the ML Platform Team